

VORBEREITUNG ZUR VORLESUNG

Handout 01 – Was ist Docker?

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Container, Images, der Docker-Daemon
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik

Worum geht es?

Docker verpackt eine Anwendung samt allem, was sie zum Laufen braucht (Bibliotheken, Werkzeuge, Konfiguration), in ein **Image**. Aus diesem Image startet man **Container** – isolierte, leichtgewichtige Prozesse. So läuft dieselbe Anwendung auf Ihrem Laptop, in der Cloud und beim Kommilitonen identisch. Das alte Problem „bei mir funktioniert es aber“ verschwindet.

Image, Container, Registry

- **Image**: unveränderliche Vorlage (ein „Bauplan“), aufgebaut aus **Layern** (Schichten). Jede Schicht beschreibt eine Änderung gegenüber der darunter.
- **Container**: eine laufende Instanz eines Images. Er bekommt eine dünne, beschreibbare Schicht *obendrauf*. Alles, was der Container schreibt, landet nur dort – und ist beim Löschen des Containers weg.
- **Registry**: ein Speicher für Images, aus dem Docker sie lädt. Der bekannteste ist **Docker Hub**; `python`, `postgres`, `adminer` und `ubuntu` kommen von dort.

Kernidee

Ein Image ist wie eine Programmdatei auf der Platte, ein Container wie ein daraus gestarteter Prozess. Sie können aus einem Image beliebig viele Container starten.

Warum nicht einfach eine virtuelle Maschine?

Eine **VM** bringt ein komplettes Gast-Betriebssystem mit – das kostet Gigabyte und Minuten. Container teilen sich den **Kernel des Wirts** und isolieren nur die Prozesse (über Linux-Techniken wie *namespaces* und *cgroups*). Deshalb starten sie in Sekundenbruchteilen und sind viel kleiner. Ein Container ist **keine** kleine VM, sondern ein besonders gut abgeschotteter Prozess.

Der Docker-Daemon

Wenn Sie `docker run ...` tippen, redet das `docker`-Kommando (der **Client**) mit einem Hintergrunddienst, dem **Docker-Daemon** (`dockerd`). Der Daemon lädt Images, startet Container und

verwaltet Netzwerke und Volumes. In diesem Kurs läuft der Daemon im Dev-Container (*Docker-in-Docker*) – mehr dazu im Zusatz-Handout.

Warum sind Änderungen flüchtig?

Weil die beschreibbare Schicht **zum Container gehört**, nicht zum Image. Löschen Sie den Container (etwa mit `docker rm` oder automatisch durch `--rm`), verschwindet diese Schicht. Genau das erleben Sie in Übung 01, wenn ein zweiter Ubuntu-Container nichts mehr von der vorher installierten Software weiß. Wie man Daten *trotzdem* behält, ist Thema von Übung 02.

Die wichtigsten Befehle

Befehl	Bedeutung
<code>docker run IMAGE</code>	Container aus einem Image starten
<code>docker run -it IMAGE bash</code>	interaktiv mit Shell
<code>docker run --rm IMAGE</code>	Container nach Ende automatisch löschen
<code>docker ps</code> / <code>docker ps -a</code>	laufende / alle Container anzeigen
<code>docker images</code>	vorhandene Images anzeigen
<code>docker rm</code> / <code>docker rmi</code>	Container / Image löschen